UID:

1. Algorithm

Let the input be an n-length array called coins.

CoinGame(coins):

1. Initialize an n x n size array, Opt, to store player 1's max score for a subset of coins. Set all values to 0 initially
2. Let i be the first element in the array and let j be the last element in the array.
3. For (i = 0; i < n; i++):
    a. Opt[i,i] = coins[i] // only one coin
4. For (k = 2; k <= n; k++): // for cases where 2 or more coins
    a. For (i = 0; i <= n - k; i++):
        i. j = i + k -1
        ii. Note: for below, if any out of bounds, substitute that Opt with 0 in the min
        iii. Opt[i,j] = max(coins[i] + min(Opt[i+1,j-1],Opt[i+2,j]),
                coins[j] + min(Opt[i+1, j-1],Opt[i,j-2]))
5. Output Opt[0, n-1]

notes for myself:

for picking 1 (first 1)

// p2 will try to minimize your score with what is left after you pick i: i+1 … j

- they will pick j if [i+1,j-1] results in minimized score for you
- they will pick i if [i+2,j] results in minimized score for you

Proof

Correctness:

Lemma: When there is more than one coin, the following recurrence relation holds:

Opt[i,j] = max(coins[i] + min(Opt[i+1,j-1],Opt[i+2,j]),
coins[j] + min(Opt[i+1, j-1],Opt[i,j-2]))

Base case: Let n = 1. If there is 1 element in the list, Opt[0,0] = coins[0]. We return coins[0]. This is correct since there is only one element in the list, so the only choice is to pick that coin. Thus, it is the maximum possible score.

Inductive hypothesis: Assume the algorithm and recurrence is correct in returning player 1's maximum score for all n = k.

Inductive step: Let n = k+1.

Case 1: Player picks the i index element (left most). This leaves elements i+1 to j in the array. From that, player 2 will try to minimize player 1's score. He can either pick j, leaving [i+1][j-1] for 1, or i+1, leaving [i+2][j]. Player 2 will try to play optimally by minimizing 1's earnings. So, we can add the minimum of what player 1 will earn in those two options to their picked element's value (coins[i]), as shown in the first line of the max statement. This is the net value 1 will earn from his choice.

Case 2: Player picks the j index element (right most). This leaves elements i to j-1 in the array. From that, player 2 will try to minimize player 1's score. He can either pick i, leaving [i+1][j-1], or j, leaving [i][j-2] for 1. So, we can add the minimum of what player 1 will earn in those two options to their picked elements value (coins[j]), as shown in the second line of the max statement.

From these cases, player 1 wants to pick the one that maximizes their earnings, so Opt[i,j] will be set to the max of the two cases. And since doing this for k+1 is based on the past subarrays of length k that we said were correctly calculated by our inductive hypothesis, this is correct too. In other words, we have the maximum score and are adding the maximal value we can to it, getting the overall maximum value.

Completeness: We find the maximum winning value for player 1 regardless of what actions 2 takes. We consider both choices 2 could possibly make to respond to 1 and maximize it based on that. We also consider all responses player 1 could take, so we are guaranteed to find the maximum winning value as required by the problem.

Termination: The for loop ends once we fill in the entire array. Therefore, we know there are no infinite loops. We start calculating values and then use past values to solve the entire array until the entire array is filled, ensuring we do not have infinite loops and the algorithm terminates.

Time Complexity Analysis $O(n^2) + O(n) + O(n^2) = O(n^2)$

- Step 1 is $O(n^2)$ to initialize the array and set all the values to 0 (would be two for loops).
- The base case for loop runs at most n times and then does an O(1) operation in each iteration, giving O(n).
- The k for loop runs n-1 times. For each value of k, there are n-k iterations of the inner for loop. The total number of iterations of the inner for loop for all values of k ranges from n-1 iterations, then n-2 and so on…till 1. Overall this gives $O(n^2)$ for both for loops.
- All the calculations are O(1) time (addition, min, max). In other words, everything within the nest for loops is constant time.

2.

Algorithm:

starting s – target t (v – w)

1. Let n = number of nodes of G.
2. Initialize an n x n size array Opt.
3. Set all Opt[0,s] = 0 and Opt[0,v] = inf for all nodes v not equal to s.
4. Initialize an n x n size array Count.
5. Set all values in Count to 0.
6. Set all Count[0,s] = 1, since there is only 1 path of length 0 from something to itself.
7. For (i = 1; i <= n-1; i++)
    a. For all v in V:
        i. Set Opt[i,v] = min(Opt[i-1,v], $\min_{u:(u,v)inE}$(w(u,v) + Opt[i-1,u])

            ii. If (Opt[i,v] == Opt[i-1,u] + w(u,v))
                1. Count[i,v] = Count[i,v] + Count[i-1,u]
8. Output Count[n-1, t]

Proof:
Correctness:
I used the Bellman-Ford algorithm to find the shortest path in a weighted graph (proven in class), with some modifications. The modification I made was to have another n x n size array that keeps track of the number of the shortest paths. In Count[i,j], i refers to the length of the path and j refers to the ending node. The value stored in Count[i,j] is the number of shortest paths from s to j with i edges.

We find the shortest path as we do in Bellman-Ford. If a shortest path less than or equal to the previous shortest path is found, we need to increment count to indicate this. We do so by adding the number of ways to get to the previous node (u) to the current Count[i,v]. This is demonstrated by Count[i,v] = Count[i,v] + Count[i-1,u]. Since we are using the Opt array, we ensure that at any given point, we only store the shortest path distances in Opt and the Count values only refers to the number of these shortest paths. Finally, we return Count[n-1,t], which is the total number of shortest paths from s to t in n-1 steps. (It cannot be longer than this since there are n nodes).

Completeness: The problem statement requires that we return just the number of shortest paths from v to w. Count[n-1,t] is the number of n length paths to t.

Termination: Both of my for loops have finite termination conditions, so they will not run infinitely. We will Opt and Count until they are completely filled, using the past subproblems to do so. Therefore, the algorithm terminates.

Time Complexity Analysis: O(n(n+m))
Bellman-Ford typically has a time-complexity of O(n(n+m)), as proven in class. The modification I made includes initializing and setting values in a new array (+$O(n^2)$), and O(1) operations like incrementing Count's values. O(n(n+m)) still dominates, so that is the time complexity.

3. Algorithm
WordSegmenter(y):
1. Initialize an n+1 length array, Opt and set all values to -inf.
2. Opt[0] = 0.
3. For (i = 1; i <= n; i++)
    a. For (j = 0; j < i; j++)
        i. Opt[i] = max(Opt[i], Opt[j] + quality(y_{j+1}...y_i))
4. Output Opt[n]

Proof
Correctness:

Lemma: For all string subsets that are non-empty, the following recurrence relation calculates the maximum total quality of any possible segmentation: Opt[i] = max(Opt[i], Opt[j] + quality(y_{j+1}...y_i))

Base case: When n = 0, we have only one possible segmentation–the empty string. Since there are no letters, the total quality will always be 0.

Inductive hypothesis: Assume the algorithm correctly computes the maximum possible quality for any input string of size n = k.

Inductive step: Let a string be length n = k+1. For the k+1th element, we segment the string at all possible values of j (where j ranges from 0 to i-1) to find the best option. We do this by comparing the current value of Opt[i] to the value that would result if we segmented the string at j and thus added the quality of the string after j to the optimal solution up until j. We know the solution Opt[j] is optimal since by our inductive hypothesis, we calculated it correctly, so it maximizes the total quality.

If there is no value for Opt[i] yet, Opt[i] is -inf, so the other item will be chosen as the max. Opt[i] becomes the value up till j (Opt[j]) and then the quality of the rest of the segmentation (from j+1 to i). If there is a value for Opt[i] we compare what leads to a higher overall quality: the current value of Opt[i] or the quality when we segment words at j and then add the quality of the rest of the string's remainder. This is what the max statement of the recurrence relation decides.

Completeness: As required by the problem, we compute the possible maximum quality with any segmentation. We do so by using this sort of recursion to consider every possible spot we can segment the string at. In other words, we consider all possible subsetting variations of the string.

Termination: Since the for loops have finite ending conditions of n and i, they will terminate. We only calculate each value in Opt one time, and stop once the entire array is filled. Therefore, the algorithm will terminate.

Time Complexity Analysis $O(n^2)$
Step 1 takes $O(n^2)$ time to set the values initially (would be two for loops). Outer for loop runs n times. Inner loop runs i times, for each iteration of the outer loop. When i = n, the inner loop runs n times (0,1,2…n-1). This generalizes to $O(n^2)$ overall for the for loops. (Using sum n(n+1)/2) Everything inside the for loops is constant time operations.

4. Algorithm:
take from x check left, take from y check above

Let n be the length of string s.
Let x' be repetitions of x for length n. Let y' be repetitions of y for length n.

Unraveler(s,x,y)

1. Initialize an n+1 x n+1 size array called Opt. Set all values to False initially.
2. Opt[0,0] = True
3. For (z = 1; z <= n; z++)
    a. For (i = 0; i <= n; i++)
        i. j = z - i
        ii. If Opt[i-1,j] = True and s[i+j] = x'[i]
            1. Opt[i,j] = True
        iii. Else if Opt[i,j-1] = True and s[i+j] = y'[j]
            1. Opt[i,j] = True
        iv. Else
            1. Opt[i,j] = False
4. If any Opt[i,j] where i+j = n is True, return True.
5. Else, return False.

Proof:

Correctness:

Lemma: This problem is solved by the following recurrence: Opt[i,j] = True if (1) Opt[i-1,j] = True and s[i+j] = x'[i] or (2) Opt[i,j-1] = True and s[i+j] = y'[j].

Base case: Let |s| and |x| and |y| be = 0. Opt[0,0] = 0. Since 0+0 = 0, and Opt[0,0] is true, we would return true. This is the correct output since an empty string is an interleaving of two empty strings.

Inductive hypothesis: Assume that the algorithm is correct for all string lengths of length n = p and returns whether or not the string is an interleaving of x and y.

Inductive step: Let the string s have length n = p + 1. We already have the correct boolean answer for the last p string elements, based on our inductive hypothesis. Assume there are two cases:

Case 1: The first p characters form a valid interleaving of x and y so far. So, we check the next character. If to the left of this (i-1 spot) the string was an interleaving of x so far (True), and this new element matches the next element we'd add from x', the string is still an interleaving, so we set Opt to true. If to the top of this (j-1 spot) the string was an interleaving of y so far, and this new element matches the next element we'd add from y', the string is still an interleaving, so we set Opt to true.

Case 2: The first p characters are not an interleaving of x and y so far. Thus, the last spot to the left or above (so the results from the string length p), would be false, so this would result in false too. If we do not have an interleaving so far, adding the next element from x or y would not result in one.

Finally, if any i + j that equals n results in Opt[i,j] = True, that means we found a valid interleaving in the entire string of length n. If there is no such pair, the string is not an interleaving.

Completeness: Our algorithm fulfills the requirements in the problem to find a valid interleaving of x and y. If anything besides the elements of x and y are in the string s, we would return false since an interleaving should not have any other elements. Each substring is only evaluated as true if only elements of x and y are in it thus far (so the last Opt value is true) and the next element to add is also either x or y.

Termination: Both loops have a finite number of iterations and are going to stop once the condition is no longer valid. We stop once the whole array is filled, returning true if there exists a valid interleaving throughout the string and false otherwise – ensuring termination.

Time Complexity Analysis: $O(n^2) + O(1) + O(n^2) + O(n)$ simplifies to **$O(n^2)$**
Step 1 is O(1) time to make the array and n^2 time to set each value to false initially (would be two for loops). Step 2 takes O(1) time. The outer for loop runs n times, and so does the inner one. This gives us $O(n^2)$ The operations inside the loops, filling each spot in the array, are O(1) time since we use the previous subproblems. The final check takes O(n) time. This is because we check all pairs where i + j = n and there are n+1 such pairs.

5.

a.

	Minute 1	**Minute 2**	**Minute 3**	**Minute 4**
A	2	1	10	50
B	1	1	30	20

m1: A - 2
m2: move
m3: B - 30
m4: B - 20

Correct Answer: A (2), A (1), A (10), A (50), resulting in 63 total.

What algorithm gives: A (2), move, B (30), B (20), resulting in 52 total.

The correct answer would be just staying at A throughout the whole time. The algorithm given would incorrectly switch to B at minute 2, and then stay there till the end.

b. **Algorithm:**

1. Initialize an array OptA[i] of size n and an array OptB[i] of size n.
2. Set P[0] = 0

3. Set OptA[0], OptB[0] = 0
4. If A[1] > B[1]
    a. OptA[1] = A[1]
    b. OptB[1] = 0
    c. Set P[1] = A
5. Else
    a. OptB[1] = B[1]
    b. OptA[1] = 0
    c. Set P[1] = B
6. For (i = 2; i <= n; i++)
    a. If (P[i-1] == A)
        i. OptA[i] = max(OptA[i-1]+A[i], OptB[i-2])
        ii. If OptA[i] = OptA[i-1]+A[i] // stayed at A
            1. P[i] = A
        iii. Else
            1. P[i] = move
            2. P[i+1] = B
    b. If (P[i-1] == B)
        i. OptB[i] = max(OptB[i-1]+B[i], OptA[i-2])
        ii. If OptB[i] = OptB[i-1]+B[i]
            1. P[i] = B // stayed at B
        iii. Else
            1. P[i] = move
            2. P[i+1] = A
7. Return P.

Proof:
Correctness:
Lemma: If the last machine in the algorithm was A, max(OptA[i-1]+A[i], OptB[i-2]) indicates whether to stay at A or make a move. If the last machine was B, max(OptB[i-1]+B[i], OptA[i-2]) indicates whether to stay at B or make a move.

Base case: Let n = 1. We pick whichever machine gives us the maximal value and set P[1] to that, indicating that at minute 1 we pick that machine. This is correct since there is only one minute so the optimal solution would just be to pick the maximal value, not move.

Inductive hypothesis: Assume the recurrence relation in the algorithm is correct for all n = k.

Inductive step: Let n = k+1. We have the optimal solution of the last k entries stored in OptA[k], if A was the last element, and in OptB[k], if B was the last element, as by our inductive hypothesis. There are two cases:

Case 1: We are currently on machine A. We can either stay at A, adding the next value (OptA[i-1]+A[i]) or we can move to B. If we move to B, we do not gain anything since the step is a move (OptB[i-2]).

Case 2: We are currently on machine B. We can either stay at B, adding the next value (OptB[i-1]+B[i]) or we can move to A. If we move to A, we do not gain anything since the step is a move (OptA[i-2]).

This proves our lemma, and thus the recurrence relation, true.

Completeness: The problem requires that we return an optimal plan for A and B for length n. Since we calculate the optimal moves for each subset which means it is ultimately optimal for the entire n-length sequences of A and B, we completely solve this problem.

Termination: The for loop only runs from 2 to n, so it is finite and will terminate. There are no other loops, and the algorithm ends when we fill P for length. So, the algorithm terminates.

Time complexity analysis not required.